What is RTL?

RTL (Register Transfer Level) is a way of describing a digital system in terms of:

- **Registers** (storage elements: flip-flops)
- **Combinational logic** between those registers
- **Data transfer** that happens **on clock edges**

In simple words:

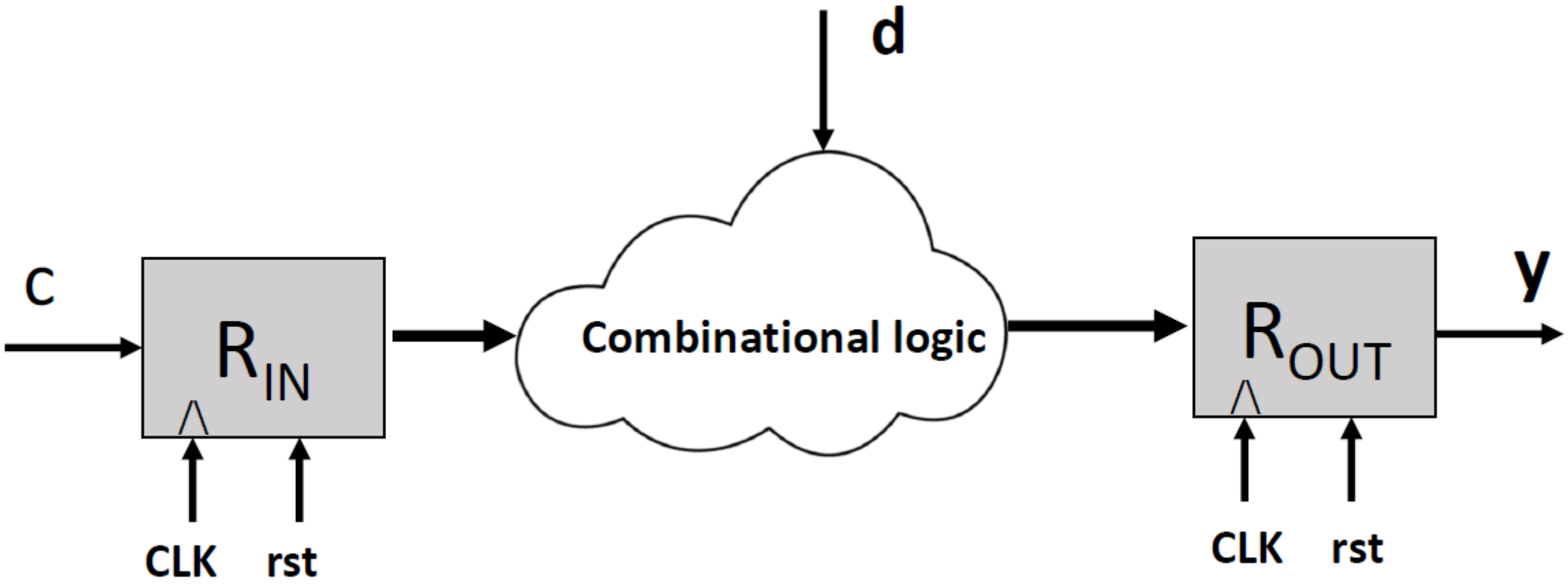
> **RTL describes how data moves from one register to another through combinational logic on each clock cycle.**

RTL view (conceptual)

```scss
Register  ──►  Combinational Logic  ──►  Register
   (FF)              (logic)               (FF)
```

- Registers update **only on clock edge**
- Combinational logic computes **between clock edges**

RTL diagram



```
module rtl_logic (
    input  wire        clk,
    input  wire        rst,
    input  wire        en,
    input  wire [7:0]  x_in,
    output reg  [15:0] y_out
);
    reg  [7:0]  x_reg;
    reg  [15:0] comb_out;

    parameter [7:0]  A = 8'd5,
    parameter [15:0] B = 16'd100

    always @(posedge clk or posedge rst) begin
        if (rst)
            x_reg <= 8'd0;
        else if (en)
            x_reg <= x_in;
    end

    always @(x_reg) begin
        comb_out = (A * x_reg) + B;
    end

    always @(posedge clk or posedge rst) begin
        if (rst)
            y_out <= 16'd0;
        else if (en)
            y_out <= comb_out;
    end
endmodule
```

RTL view (conceptual)

```scss
Register  ──►  Combinational Logic  ──►  Register
   (FF)              (logic)               (FF)
```

- Registers update **only on clock edge**
- Combinational logic computes **between clock edges**

RTL is a hardware modelling style where registers are updated on clock edges, and data transfer between registers is described using combinational logic, typically implemented using multiple always blocks in Verilog.

One `always @(posedge clk)` = registers

One `always @(...)` without clock = combinational logic

RTL = registers + combinational logic between them

Question:

Design an **arbitrary sequence counter** in Verilog RTL. The counter must cycle through the following sequence on every rising edge of `clk`:

3 → 7 → 1 → 12 → 4 → 9 → 3 → ...

Include an active-high reset `rst` that forces the counter output to **3**.

```
5  module arb_seq_counter (
6      input  wire       clk,
7      input  wire       rst,        // async, active-high
8      output reg  [3:0] count
9  );
10
11     reg [3:0] next_state;
12
13     // Next-state (combinational) logic
14     always @(*) begin
15         case (count)
16             4'd3:   next_state = 4'd7;
17             4'd7:   next_state = 4'd1;
18             4'd1:   next_state = 4'd12;
19             4'd12:  next_state = 4'd4;
20             4'd4:   next_state = 4'd9;
21             4'd9:   next_state = 4'd3;
22             default: next_state = 4'd3;  // recovery
23         endcase
24     end
25
26     // State register with asynchronous reset
27     always @(posedge clk or posedge rst) begin
28         if (rst)
29             count <= 4'd3;
30         else
31             count <= next_state;
32     end
33
34 endmodule
```

Problem Statement

- Design a digital system that begins operation when a **`go`** signal is triggered to logic ‘1’. After a **5-second delay**, the system enters an **active state**.
- In the active state, if the **`mode = 01`** signal is pressed, the system transitions to an **addition state**, where it computes the sum of two **4-bit inputs (a and b)**. If the **`mode=10`** signal is asserted, the system transitions to a **multiplication state**. Mode=00 or 11 is invalid. An led should blink with 1 hz frequency to notify output is invalid.
- In the active state, if the **`stop`** signal is asserted, the system returns to the **reset (idle) state after 5 seconds delay**.
- Put some status signal ‘idle’, set it to high when system is in reset state, use ‘active’ output signal to notify user that circuit is in active computation mode where user can do add or mult operation.

Assume that a clock signal with 1 Hz frequency is available.

Design and implement a **clock frequency divider** circuit that converts a high-frequency input clock into a lower-frequency clock using **Verilog RTL**. Verify the design through simulation and (optional) implement on an FPGA board.

Given

- Input clock: `clk_in` (assume **100 MHz**)
- Reset: `rst` (active-high)
- Target output clock: `clk_out` = **1 Hz** (you can change this to 1 Hz / 1 kHz etc.)